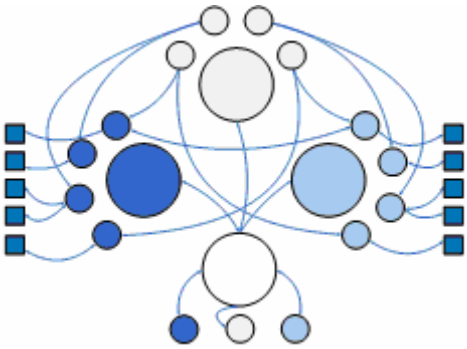


框架概览与 UML 图解

学习使用 PureMVC 构建
健壮、可扩展、可维护的应用程序

框架概览

本文讨论 PureMVC 框架的类和接口，并通过简单的 UML（统一建模语言）图，说明它们各自的角色、职责与协作关系。

PureMVC 框架的目标非常明确：帮助你把应用程序中的编码关注点分离为三个独立的层，即 Model（模型）、View（视图）和 Controller（控制器）。

在这一经典 MVC 设计元模式的实现中，应用程序的三层由三个单例表示。单例是只能创建一个实例的类。

第四个单例 Facade（外观）为整个应用程序的通信提供统一接口，从而简化开发。

Model 按名称缓存 Proxy 的引用。Proxy 公开操作数据模型的 API，包括操作从远程服务获取的数据。

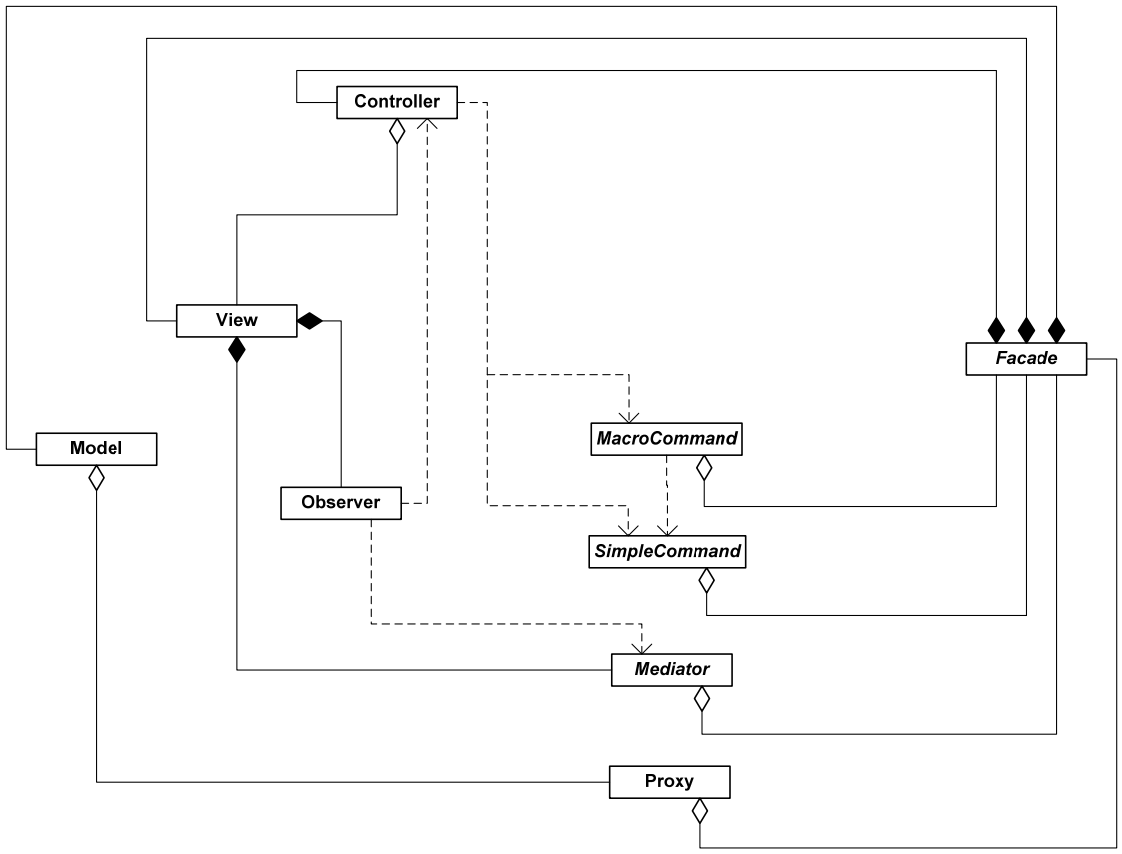
View 主要按名称缓存 Mediator 的引用。Mediator 负责适配和管理组成用户界面的视图组件。

Controller 维护名称到 Command 类的映射。Command 无状态，只在需要时创建。

Facade 初始化并缓存核心角色（Model、View 和 Controller），并提供访问它们全部公共方法的统一入口。

作者：Cliff Hall <cliff@puremvc.org>
最后修改：2008 年 3 月 5 日

Facade 与核心角色



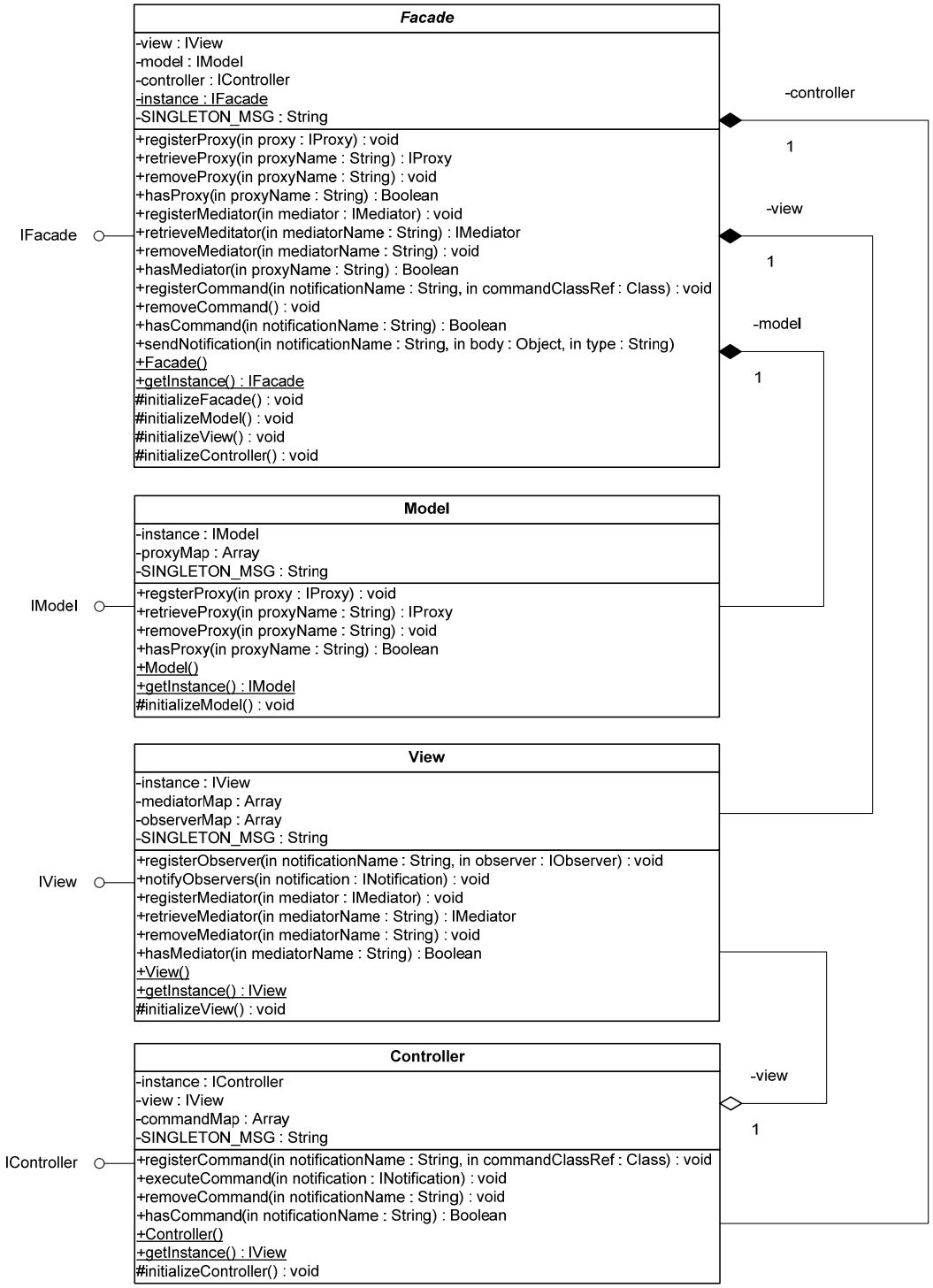
Facade 类让构成应用程序主体的 Proxy、Mediator 和 Command 能够以松耦合的方式相互通信，无须导入核心框架角色，也不必直接与它们交互。

当我们为应用程序创建具体的 Facade 实现后，在与 Facade 交互的同时，就能直接使用开箱即用的核心角色。这把开发者成功使用框架所需掌握的 API 知识降到了最低。

核心角色 Model、View 和 Controller 分别实现 IModel、IView 和 IController 接口。Facade 实现 IFacade，后者通过组合涵盖全部核心接口。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。
PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Facade 与核心角色



PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

View、Mediator 与视图组件

View 类实现为一个单例，用于缓存具体的 IMediator 实例，并提供对它们的访问。

Mediator 帮助我们创建或复用现有用户界面组件，而无须让组件了解与之通信的 PureMVC 应用程序。具体的 Mediator 必须实现 IMediator 接口，通常通过继承框架的 Mediator 类来完成。

视图组件负责显示数据或接受用户操作。在基于 Flash 的应用程序中，它们通常使用事件与 Mediator 通信，并公开一些属性，供具体的 Mediator 检查或管理。Mediator 将视图组件与其数据连接起来，并代表组件与系统其他部分通信。

具体的 Mediator 注册到 View 时，会被询问对哪些通知感兴趣。它必须返回一个 Array，包含希望接收的全部通知名称。

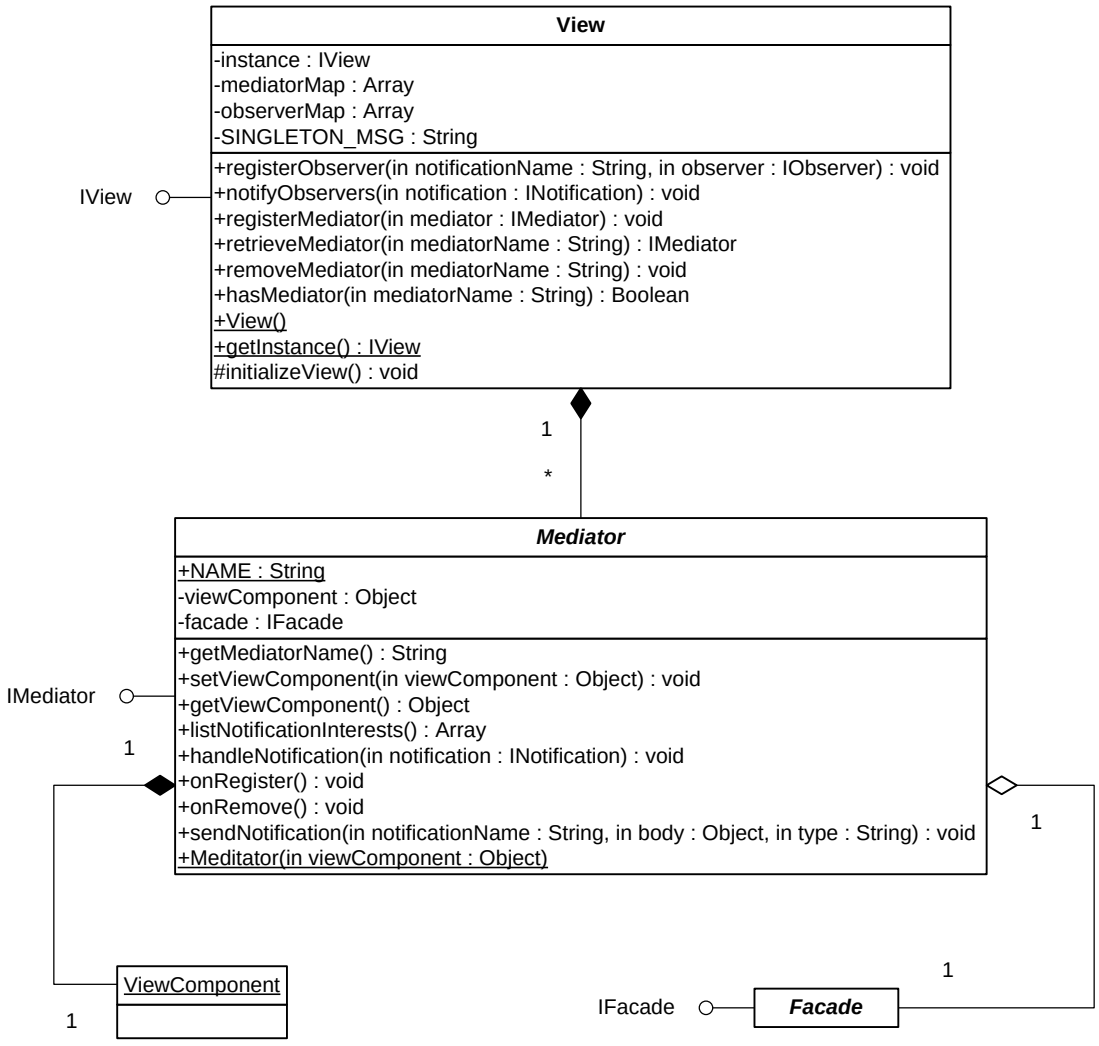
由于必须实现 IMediator 接口，具体的 Mediator 会有一个 handleNotification 方法。它注册到 View 时，数组中的每条通知都会对应创建并注册一个 Observer 实例。这样，只要发出 Mediator 关注的通知，它的 handleNotification 方法就会被调用。

框架的 Mediator 类实现了 INotifier，因此拥有 sendNotification 方法。该方法接收新通知的参数，构造 Notification，并使用 IFacade 单例发送它。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

View、Mediator 与视图组件

Mediator 的受保护 facade 属性被初始化为已注册的 IFacade 实例。因此，必须先初始化应用程序的具体 Facade，再构造 Mediator。



PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Model、Proxy 与数据对象

Model 类实现为一个单例，用于缓存具体的 IProxy 实例，并提供对它们的访问。

Proxy 帮助我们向应用程序公开数据结构、实体类，以及支持它们的领域逻辑与服务，使这些内容易于在其他地方复用，或在重构时尽量减少对应用程序其他部分的影响。

具体的 Proxy 可以只管理一个本地数据对象的引用。在这种情况下，与它交互的惯用法可能包括同步地设置和获取数据。

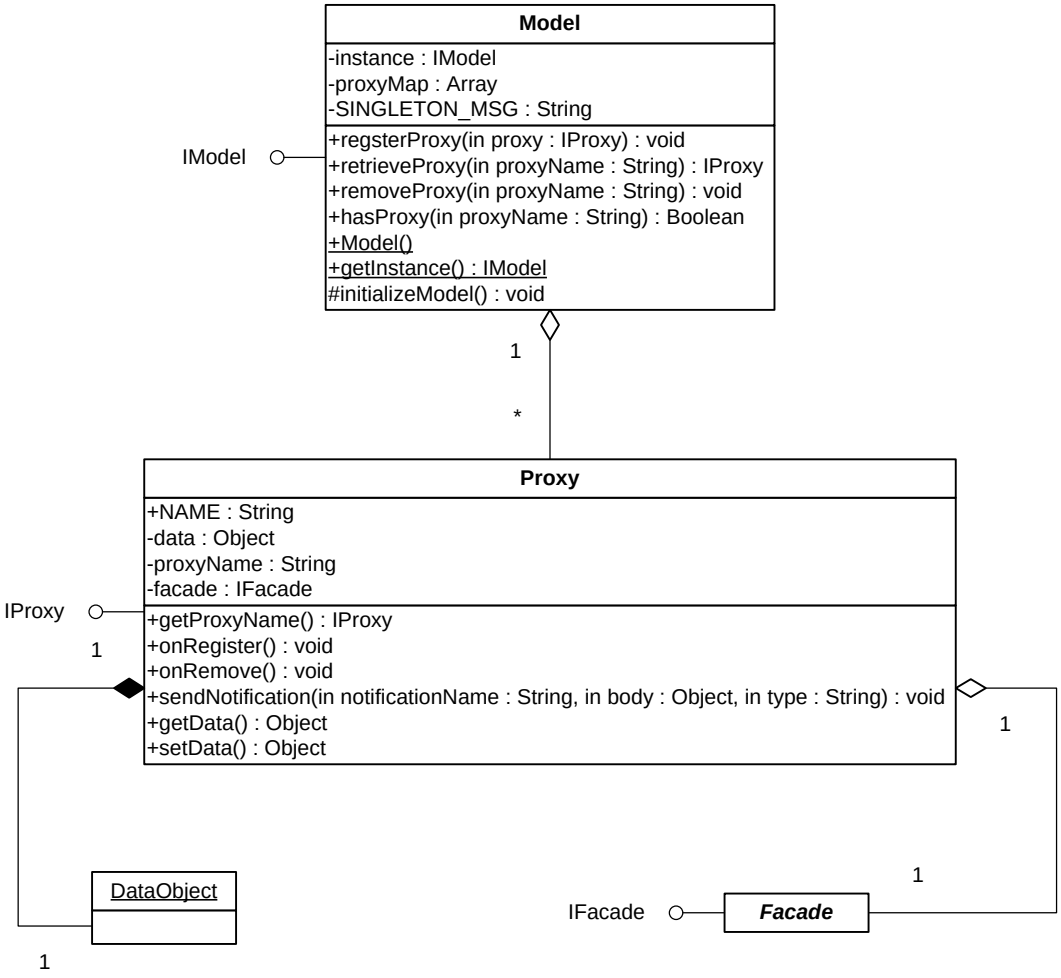
Proxy 也可以封装应用程序与远程服务之间的交互，以保存或获取数据。此时，调用方可以调用 Proxy 的方法或设置它的数据，然后等待 Proxy 从服务获取数据后发出的通知。

框架的 Proxy 类实现了 INotifier，因此拥有 sendNotification 方法。该方法接收新通知的参数，构造 Notification，并使用 IFacade 单例发送它。

其受保护 facade 属性被初始化为已注册的 IFacade 实例。因此，必须先初始化应用程序的具体 Facade，再构造 Proxy。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。
PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Model、Proxy 与数据对象



PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。
PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Controller 与 Command

Controller 类实现为一个单例，维护通知名称与 Command 类引用之间的映射。

Command 可以获取 Proxy 并与其交互，与 Mediator 通信，或执行其他命令。命令通常用于编排复杂操作或应用程序启动、关闭等系统级活动。

Controller 初始化时，通常由一个 IFacade 实现触发，会为每个“通知到命令”的映射创建适当的 Observer 实例，并向 View 注册。这样，只要广播任何已注册的通知，就会调用 Controller 的 executeCommand 方法，并把通知传入。

View 广播通知时，Controller 会实例化对应的 Command 类，调用其 execute 方法，并传入通知。

PureMVC 包含两种易于扩展的 ICommand 类实现。两者都实现 INotifier，因此都具有 sendNotification 方法，以及一个初始化为 IFacade 单例的受保护 facade 属性。

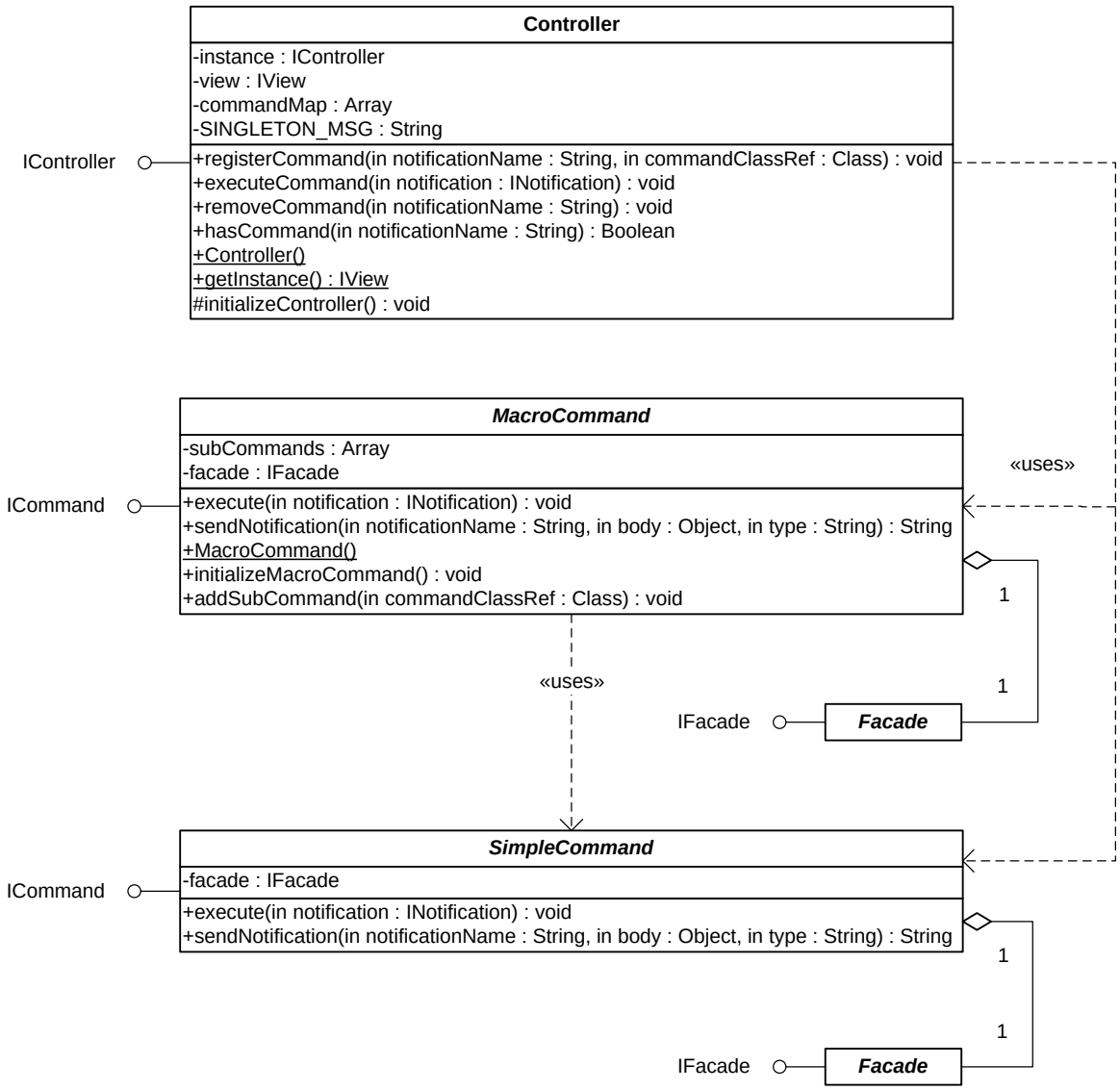
SimpleCommand 类只提供一个 execute 方法，调用时传入 Notification 对象。

MacroCommand 类允许按顺序执行多个“子命令”。每个子命令都会被创建，其 execute 方法将接收到原始通知。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Controller 与 Command

MacroCommand 在构造函数中调用 initializeMacroCommand 方法。你可以在子类中重写该方法，为每个需要添加的命令调用一次 addSubCommand。添加的可以是 SimpleCommand，也可以是其他 MacroCommand。



PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

View、Observer 与 Notification

Proxy、Mediator 和 Command 通过广播通知，以松耦合且不依赖平台的方式相互通信。

- Proxy 可以广播通知，但不监听通知。
- Mediator 监听通知，也可以广播通知。
- Command 由通知触发，也可以广播通知。

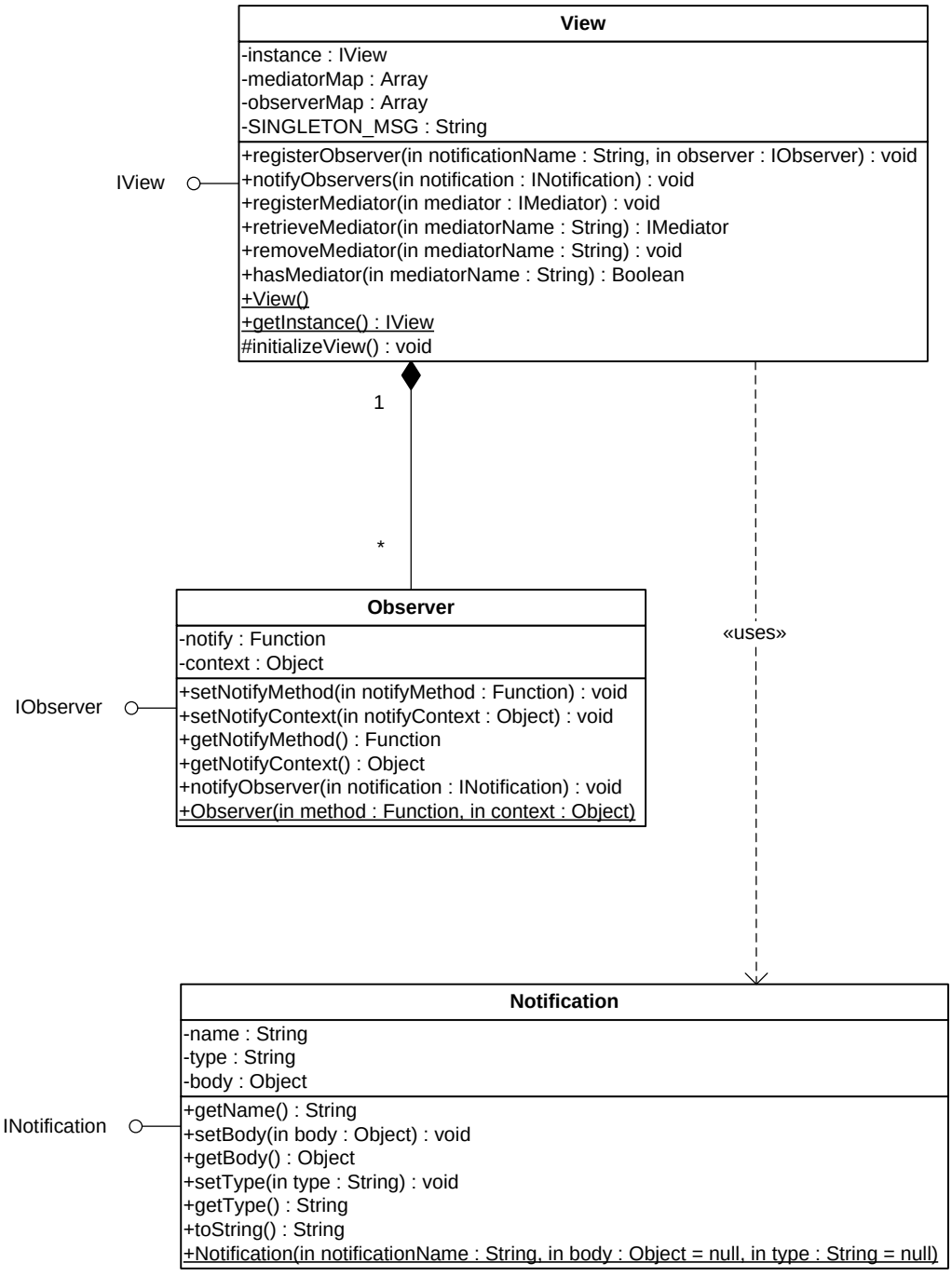
PureMVC 应用程序也可能运行在没有底层 flash.events.Event 和 EventDispatcher 类的纯 ActionScript 环境中。因此，框架实现了一套基于观察者的通知机制，使核心角色能够与系统其他部分通信。

为此，PureMVC 使用了观察者模式。IObserver 实例持有一个希望收到通知的对象引用，即“通知上下文”，以及该对象上在 INotification 广播时要调用的方法，即“通知方法”。

View 负责管理通知名称到 Observer 列表的映射，并在通知发出时通知所有相关观察者。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

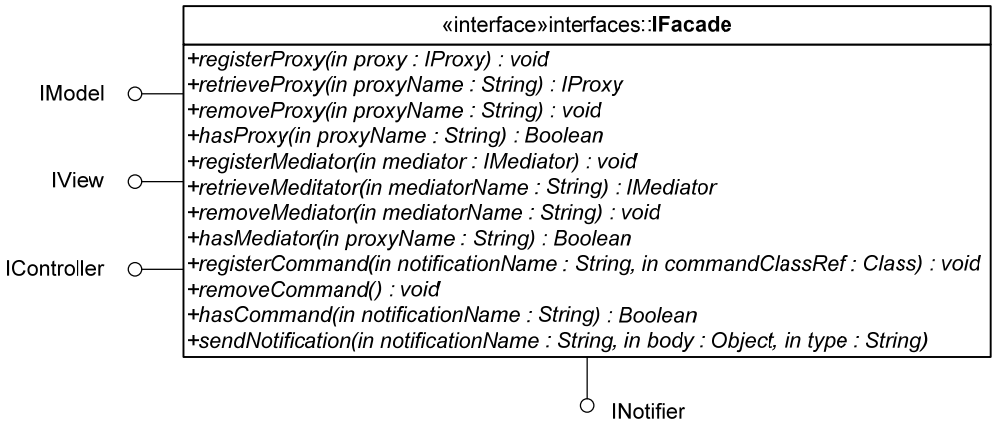
View、Observer 与 Notification



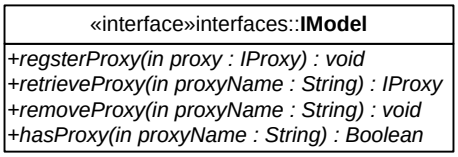
PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。
PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

接口

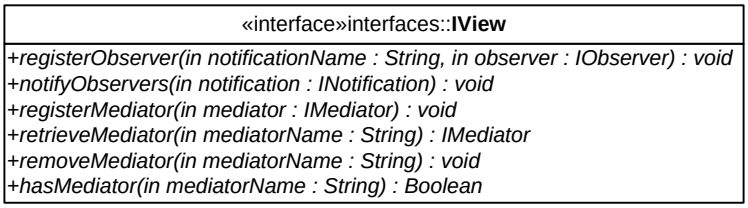
IFacade



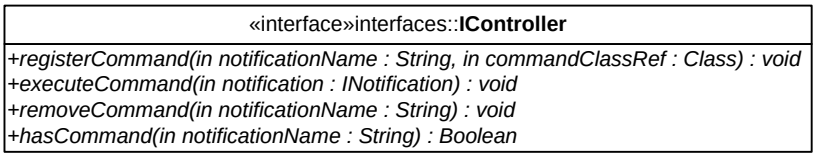
IModel



IView



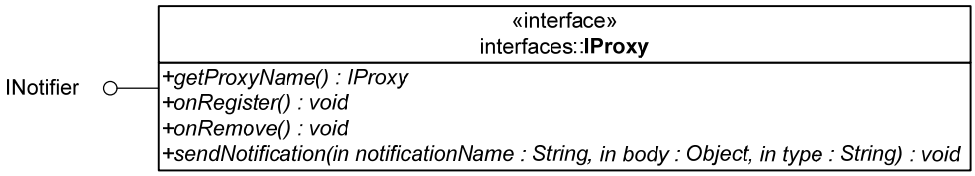
IController



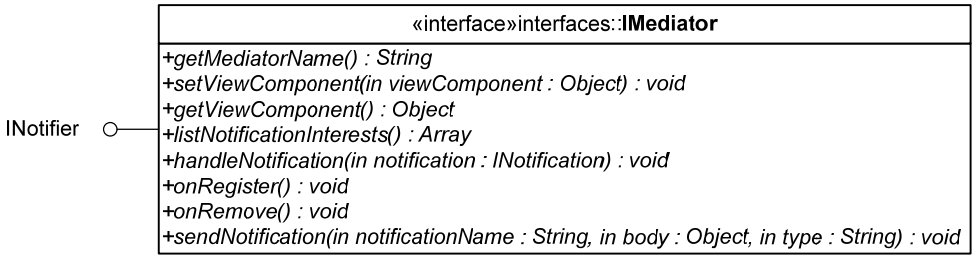
PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

接口

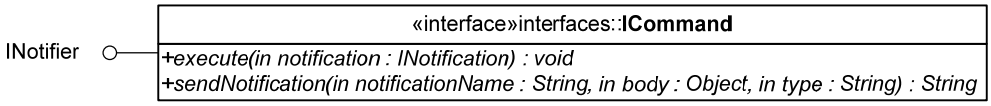
IProxy



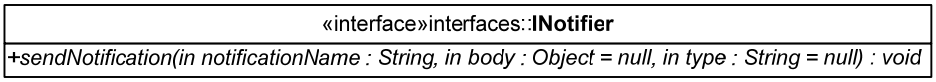
IMediator



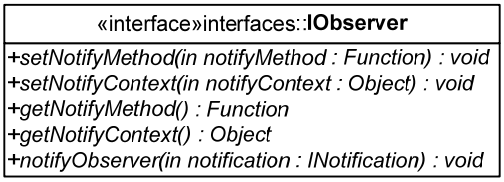
ICommand



INotifier



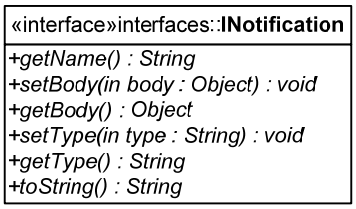
IObserver



PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。
PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

接口

INotification



PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006-08，保留部分权利。再使用须遵守知识共享署名 3.0 未本地化版本许可协议。
PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。